



Dance4Life

Envelhecimento activo

- Aprendizagem por Reforço-

Mestrado em Inteligência Artificial

Carlos Bergueira
Diego Silva
Filipa Pereira
Rui Rodrigues

PG11605
PG59999
PG42201
PG 7942

Universidade do Minho | Junho 2026

Dance4Life Reinforcement Learning Coach

Este projeto implementa um sistema de **Reinforcement Learning (RL)** para o ecossistema Dance4Life, permitindo treinar agentes inteligentes capazes de adaptar intervenções de coaching com o objetivo de aumentar a atividade física dos utilizadores sem provocar níveis excessivos de fadiga ou irritação.

O projeto suporta dois algoritmos de Reinforcement Learning:

- **PPO (Proximal Policy Optimization)**
- **DQN (Deep Q-Network)**

Após o treino, os modelos podem ser exportados para **ONNX** e integrados diretamente na aplicação Android para inferência local.

Reinforcement Learning Concepts

Agente

- O agente é a entidade responsável por tomar decisões e executar ações
 - No Dance4Life, o agente representa o sistema de coaching inteligente que decide qual o nível de intervenção mais adequado para o utilizador

Ambiente

- O ambiente representa o mundo onde o agente opera
- Sempre que o agente executa uma ação, o ambiente responde alterando o seu estado e devolvendo uma recompensa
 - No Dance4Life, o ambiente simula o comportamento de um utilizador ao longo do tempo

Estado

- O estado representa a situação atual observada pelo agente
- É a informação utilizada para decidir qual a próxima ação
- O estado do ambiente é representado por:
 - [activity_level, physical_fatigue, irritation_level] exemplo [6, 2, 1]

Ação

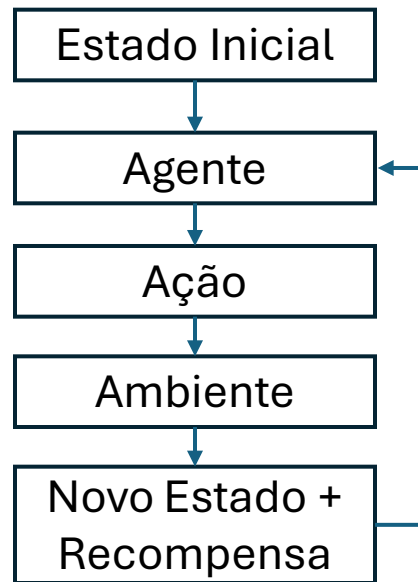
- Uma ação corresponde a uma decisão tomada pelo agente
- Após observar o estado atual, o agente escolhe uma ação que considera mais vantajosa

Recompensa

- A recompensa é o feedback fornecido pelo ambiente após a execução de uma ação.
- O objetivo do treino é maximizar a soma das recompensas recebidas ao longo do tempo
- A recompensa funciona como um mecanismo de aprendizagem que orienta o agente para comportamentos desejáveis

Reinforcement Learning Implementation

Learning Cycle



Action Space

O Agente pode escolher uma das seguintes ações

Action ID	Descrição
0	Silence
1	Low Coaching
2	Medium Coaching
3	High Coaching

Reward Function

Recompensa Positiva

+ 15 quando:

$\text{activity_level} \in [4,7]$

$\text{physical_fatigue} < 7$

$\text{irritation_level} < 5$

Penalização por Baixa Atividade

- 5 quando:

$\text{activity_level} < 2$

Penalização por Estado Crítico

-50 quando:

$\text{physical_fatigue} \geq 10$

Ou

$\text{irritation_level} \geq 10$

Supported Algorithms

PPO (Proximal Policy Optimization)

- O PPO é um algoritmo baseado em Policy Gradient.
- Em vez de aprender diretamente o valor de cada ação, aprende uma política que produz probabilidades para cada ação disponível.
- Vantagens
 - Treino estável
 - Boa capacidade de generalização
 - Menor sensibilidade a hiperparâmetros
 - Excelente desempenho em ambientes complexos
- Funcionamento
 - O agente interage com o ambiente.
 - São recolhidas trajetórias de experiência.
 - É calculada a vantagem (Advantage Estimate).
 - A política é atualizada através de clipping.
 - O processo repete-se até convergência.

DQN (Deep Q-Network)

- O DQN é um algoritmo baseado em Value Functions.
- Aprende uma função $Q(\text{state}, \text{action})$, que estima a recompensa futura esperada para cada ação.
- Vantagens
 - Simples de interpretar
 - Muito eficaz para ações discretas
 - Treino rápido
- Funcionamento
 - O agente observa um estado.
 - A rede calcula os valores Q para cada ação.
 - É escolhida a ação com maior valor esperado.
 - A experiência é armazenada num Replay Buffer.
 - O treino é realizado através de amostragem aleatória desse buffer.

Training Configuration (training_config.yaml)

- PPO Configuration
 - learning_rate: 0.0003 #Taxa de aprendizagem
 - n_steps: 1024 #Passos recolhidos antes da atualização
 - batch_size: 64 #Tamanho do mini-batch
 - n_epochs: 10 #Número de épocas
 - gamma: 0.99 #Discount factor
 - gae_lambda: 0.95 #Generalized Advantage Estimation
 - clip_range: 0.2 #Limite de clipping PPO
 - ent_coef: 0.01 #Incentivo à exploração
 - vf_coef: 0.5 #Peso da Value Function
 - max_grad_norm: 0.5 #Gradient clipping

Training Configuration (training_config.yaml)

- DQN Configuration
 - learning_rate: 0.0005 #Taxa de aprendizagem
 - buffer_size: 100000 #Replay Buffer Size
 - learning_starts: 5000 #Passos antes de iniciar treino
 - batch_size: 128 #Tamanho do batch
 - tau: 1.0 #Soft Update Factor
 - gamma: 0.99 #Discount Factor
 - train_freq: 4 #Frequência de treino
 - gradient_steps: 1 #Atualizações por ciclo
 - target_update_interval: 1000 #Atualização da Target Network
 - exploration_fraction: 0.25 #Percentagem de exploração
 - exploration_initial_eps: 1.0 #Epsilon inicial
 - exploration_final_eps: 0.05 #Epsilon final
 - max_grad_norm: 10.0 #Gradient clipping

Training Configuration (training_config.yaml)

- Global Training Parameters
 - total_timesteps: 400000 #Número total de interações
 - n_envs: 8 #Ambientes paralelos
 - eval_freq: 10000 #Frequência de avaliação
 - n_eval_episodes: 20 #Episódios de avaliação
 - seed: 42 #Seed de reprodutibilidade
 - episode_length: 96 #Duração máxima do episódio

Training

```
def _train_one(algo: str, config: dict, timesteps: int) -> None:
    algo_cfg = config[algo]
    train_cfg = config["training"]
    out_cfg = config["output"]

    seed = train_cfg["seed"]
    episode_length = train_cfg["episode_length"]
    n_envs = train_cfg["n_envs"] if algo == "ppo" else 1

    base_model_dir = Path(out_cfg["model_dir"])
    base_log_dir = Path(out_cfg["log_dir"])
    experiment_tag = out_cfg["experiment_tag"]

    model_dir = base_model_dir / algo
    log_dir = base_log_dir / algo
    model_name = f"{experiment_tag}_{algo}"

    model_dir.mkdir(parents=True, exist_ok=True)
    log_dir.mkdir(parents=True, exist_ok=True)
```

```
print(f"[dance4life] Training {model_name} for {timesteps:,} timesteps...")
print(f"  n_envs={n_envs}, seed={seed}, episode_length={episode_length}")
```

```
if algo == "ppo":
    train_env = make_envs(n_envs, episode_length, seed)
    eval_env = make_envs(1, episode_length, seed + 1)
else:
    train_env = MovementEnv(episode_length=episode_length)
    eval_env = MovementEnv(episode_length=episode_length)

model = _build_model(algo, algo_cfg, train_env, seed, log_dir)

eval_freq = train_cfg["eval_freq"]
if algo == "ppo":
    eval_freq = max(eval_freq // n_envs, 1)

checkpoint_cb = CheckpointCallback(
    save_freq=max(eval_freq, 1),
    save_path=str(model_dir / "checkpoints"),
    name_prefix=model_name,
    verbose=1,
)

eval_cb = EvalCallback(
    eval_env,
    best_model_save_path=str(model_dir / "best"),
    log_path=str(log_dir),
    eval_freq=max(eval_freq, 1),
    n_eval_episodes=train_cfg["n_eval_episodes"],
    deterministic=True,
    verbose=1,
)

model.learn(
    total_timesteps=timesteps,
    callback=[checkpoint_cb, eval_cb],
    progress_bar=True,
)

final_path = model_dir / f"{model_name}_final"
model.save(str(final_path))
print(f"[dance4life] Saved final model to {final_path}.zip")
```

- Train PPO
 - python src/train.py --algo ppo
- Train DQN
 - python src/train.py --algo dqn
- Train Both
 - python src/train.py --config config/training_config.yaml --algo both
- Training Outputs
 - Os modelos treinados são armazenados em:
 - training/checkpoints/

Evaluation

```
def evaluate(algo: str, model_path: str, n_episodes: int, render: bool) -> dict[str, Any]:
    print(f"[dance4life] Loading {algo.upper()} model from {model_path}...")
    model = _load_model(algo, model_path)

    env = MovementEnv()
    episode_rewards: list[float] = []
    episode_lengths: list[int] = []
    survived_episodes: int = 0
    action_counts: Counter = Counter()

    for ep in range(n_episodes):
        obs, _ = env.reset()
        total_reward = 0.0
        steps = 0
        done = False
        survived = True

        while not done:
            action, _ = model.predict(obs, deterministic=True)
            action = int(action)
            obs, reward, terminated, truncated, info = env.step(action)
            total_reward += float(reward)
            action_counts[action] += 1
            steps += 1
            done = terminated or truncated

            if terminated and info.get("game_over", False):
                survived = False

        if render:
            _render_step(env, action, float(reward))
```

```
        episode_rewards.append(total_reward)
        episode_lengths.append(steps)
        if survived:
            survived_episodes += 1

    if not render:
        status = "survived" if survived else "game-over"
        print(
            f" Episode {ep + 1:3d}: reward={total_reward:7.2f} "
            f"steps={steps:3d} status={status}"
        )

    env.close()

    total_actions = sum(action_counts.values())
    survival_rate = survived_episodes / max(n_episodes, 1)

    summary = {
        "algo": algo,
        "episodes": n_episodes,
        "mean_reward": float(np.mean(episode_rewards)),
        "std_reward": float(np.std(episode_rewards)),
        "min_reward": float(np.min(episode_rewards)),
        "max_reward": float(np.max(episode_rewards)),
        "mean_episode_length": float(np.mean(episode_lengths)),
        "survival_rate": float(survival_rate),
        "action_counts": action_counts,
        "total_actions": total_actions,
    }

    _print_summary(summary)

    return summary
```

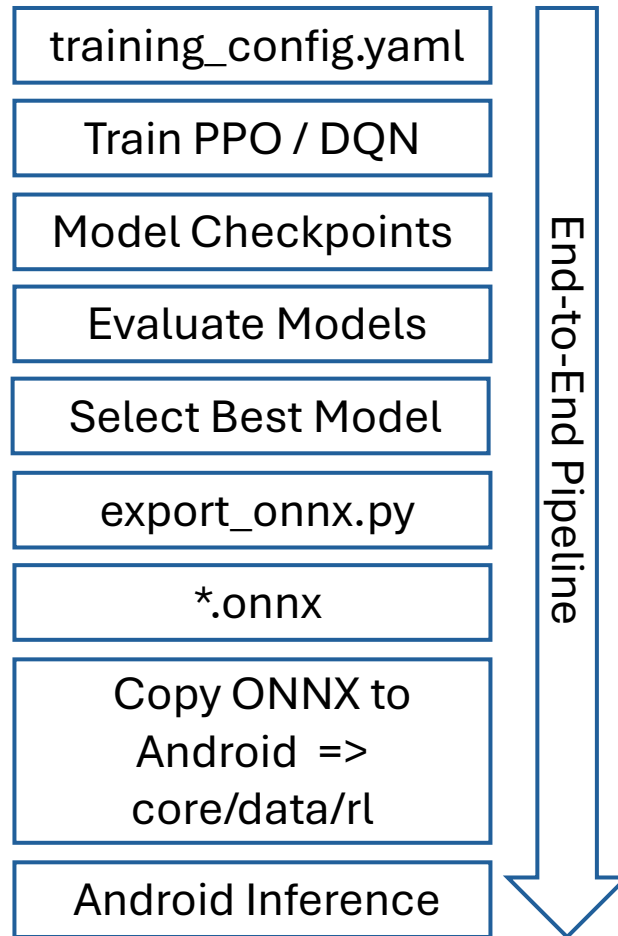
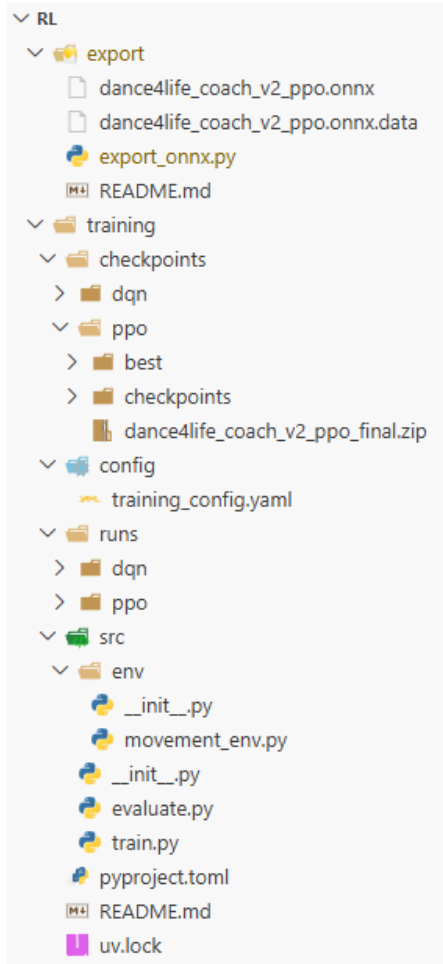
- Evaluation

- Comparação entre PPO e DQN:
 - python src/evaluate.py -- algo compare --episodes 20
- Avaliação individual:
 - python src/evaluate.py -- algo ppo --model checkpoints/ppo/best/best_model
 - python src/evaluate.py -- algo dqn --model checkpoints/dqn/best/best_model

ONNX Export/Android Integration

- Após o treino, os modelos Stable-Baselines3 devem ser convertidos para ONNX para utilização na aplicação Android.
 - `export/export_onnx.py`
 - Carrega um modelo PPO ou DQN treinado.
 - Extrai a rede de inferência.
 - Converte a rede para ONNX.
 - Gera um ficheiro compatível com ONNX Runtime.
 - Opcionalmente valida o modelo exportado.
 - O objetivo é remover a dependência de Python e Stable-Baselines3 em ambiente mobile.
- ONNX Inputs
 - `[activity_level, physical_fatigue, irritation_level]`
- ONNX Outputs
 - `[action_scores]`
 - 0 = Silence
 - 1 = Low Coaching
 - 2 = Medium Coaching
 - 3 = High Coaching
- Android Integration
 - Após a exportação, o ficheiro ONNX deve ser copiado manualmente para:
 - `Dance4Life\AndroidSensor\core\src\main\java\com\dance4life\core\data\rl`
 - A aplicação Android utiliza este modelo para executar inferência local através de ONNX Runtime, sem necessidade de Python ou Stable-Baselines3.

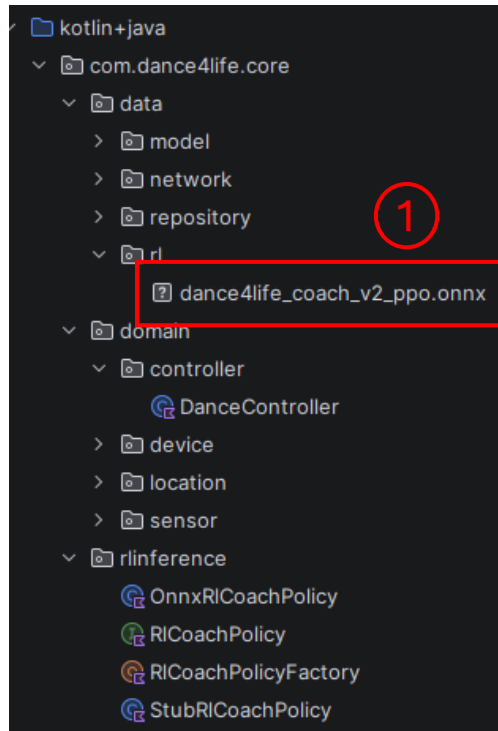
Reinforcement Learning Files, Pipeline, Technology Stack



Technology Stack:

Python 3.x
Stable-Baselines3
PyTorch
Gymnasium
ONNX
ONNX Runtime
Android
Kotlin
ONNX Runtime Mobile

Reinforcement Learning Android implementation



```
class OnnxRLCoachPolicy(
    context: Context,
    modelAssetPath: String = DEFAULT_MODEL_ASSET_PATH,
) : RLCoachPolicy {

    2 Usages
    private val ortEnvironment: OrtEnvironment = OrtEnvironment.getEnvironment()
    3 Usages
    private val ortSession: OrtSession
    2 Usages
    private val inputName: String

    init {
        val modelFile = copyAssetToCache(context.applicationContext, modelAssetPath)
        ortSession = ortEnvironment.createSession(modelPath = modelFile.absolutePath, options = OrtSession.SessionOptions())
        inputName = ortSession.inputNames.first()
    }

    override fun recommend(observation: MovementObservation): MovementRecommendation {
        val input = floatArrayOf(
            observation.activityLevel.coerceIn(0f, 10f),
            observation.physicalFatigue.coerceIn(0f, 10f),
            observation.irritationLevel.coerceIn(0f, 10f),
        )

        val action = inferAction(input)
        return toRecommendation(action)
    }
}
```

2

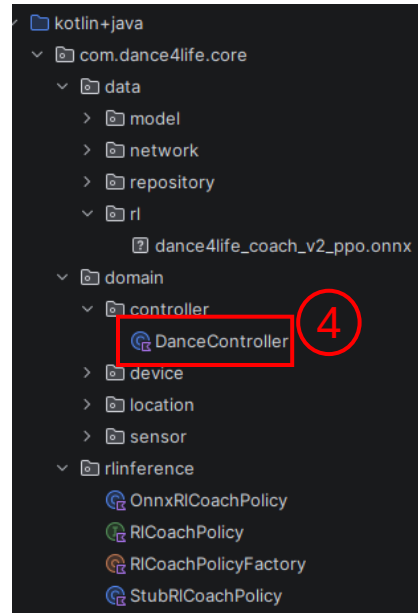
3

1 – Modelo treinado

2 – Load do modelo e criação do ambiente

3 – Invocação do modelo

Reinforcement Learning Android implementation

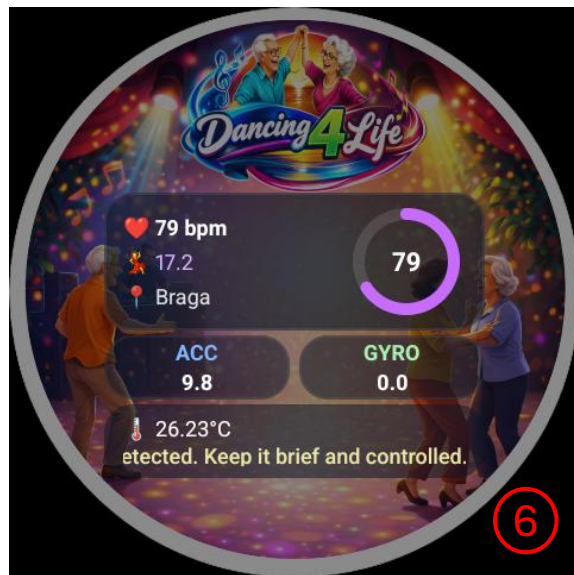


4 – Captura de evento e invocação do modelo

```
private fun calcularRitmoLocal() {
    val result = ritmoCalculator.calcular(accData, gyroData, hrData)
    lastResult = result
    onRitmoCalculated?.invoke(result.ritmo)
    val now = System.currentTimeMillis()
    ritmoBuffer.add(Pair(now, result.ritmo))
    ritmoBuffer.removeAll { now - it.first > RITMO_WINDOW_MS }
    val avgRitmo = if (ritmoBuffer.isNotEmpty()) ritmoBuffer.map { it.second }.average() else result.ritmo

    val sedentaryMinutesToday = estimateSedentaryMinutes(result.avgAcc)
    val energyLevel = estimateEnergyLevel(result.avgHR)
    val elapsedSimMinutes = ((System.currentTimeMillis() - sessionStartMs) / 1000f * SIM_MINUTES_PER_SECOND).toInt()
    val mobilityConfidence = estimateMobility(result.avgGyro)
    tickIrritationGrowthByRitmo(currentRitmo = result.ritmo)
    val activityLevel = (avgRitmo / RITMO_ACTIVITY_SCALE).toFloat().coerceIn(0f, 10f)
    val simSedentary = (sedentaryMinutesToday + elapsedSimMinutes).coerceIn(0, 480)
    val baselinePhysicalFatigue =
        ((simSedentary / 480f) * 5f +
            ((10 - energyLevel.coerceIn(0, 10)) / 10f) * 5f)
            .coerceIn(0f, 10f)
    val irritationMetric = getIrritationLevel().toFloat().coerceIn(0f, 10f)
    val physicalFatigue = maxOf(a = baselinePhysicalFatigue, b = irritationMetric)
    val observation = MovementObservation(
        activityLevel = activityLevel,
        physicalFatigue = physicalFatigue,
        irritationLevel = irritationMetric,
    )
    val recommendation = rlCoachPolicy.recommend(observation)
    onMovementRecommendation?.invoke(recommendation)
}
```

Reinforcement Learning Android visualization



- 3 – Invocação do modelo
- 5 – Resultado da Inferência
- 6 – Dados a apresentar

```
override fun recommend(observation: MovementObservation): MovementRecommendation {
    val input = floatArrayOf(
        observation.activityLevel.coerceIn(0f, 10f),
        observation.physicalFatigue.coerceIn(0f, 10f),
        observation.irritationLevel.coerceIn(0f, 10f),
    )

    val action = inferAction(input)
    return toRecommendation(action)
}
```

```
private fun inferAction(input: FloatArray): Int {
    val shape = longArrayOf(1, 3)
    val inputTensor = OnnxTensor.createTensor(env = ortEnvironment, data = FloatBuffer.wrap(array = input), shape)

    inputTensor.use { tensor ->
        ortSession.run(inputs = mapOf(inputName to tensor)).use { output ->
            @Suppress("unchecked_cast")
            val logits = (output[0].value as Array<FloatArray>)[0]
            return logits.indices.maxByOrNull { logits[it] } ?: 0
        }
    }
}
```

```
private fun toRecommendation(action: Int): MovementRecommendation {
    return when (action) {
        0 -> MovementRecommendation(
            actionId = "silence",
            title = "Silent recovery window",
            durationMinutes = 1,
            encouragementMessage = "Recovery is part of progress. We will check in again soon.",
        )

        1 -> MovementRecommendation(
            actionId = "low_intensity",
            title = "Low-intensity movement cue",
            durationMinutes = 2,
            encouragementMessage = "A gentle move keeps momentum without overload.",
        )

        2 -> MovementRecommendation(
            actionId = "medium_intensity",
            title = "Moderate coaching suggestion",
            durationMinutes = 5,
            encouragementMessage = "Great balance: this intensity supports healthy activity.",
        )

        else -> MovementRecommendation(
            actionId = "high_intensity",
            title = "High-intensity challenge",
            durationMinutes = 10,
            encouragementMessage = "Strong push detected. Keep it brief and controlled.",
        )
    }
}
```